

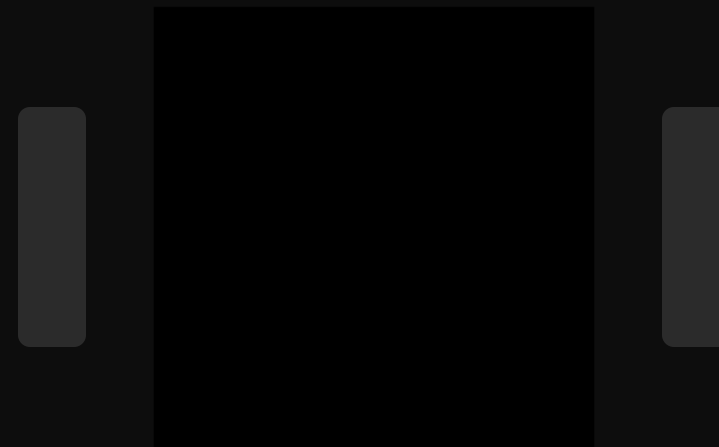
AI WORKSHOP

HOW TO VIBE-CODE

Tự tay làm ra một app — không cần biết code

10:00 · Thứ Sáu 04/09 · Phòng Cambridge Alex Pham & Triết · AI Vibe Code Trainer · TA & EB Team

KHỞI ĐỘNG



ĐÂY LÀ GAME GÌ?

BẤM ĐỂ LỘ ĐÁP ÁN

AGENDA

Hai tiếng hôm nay

- 1

Vì sao ai cũng học được

Case study: con survey chatbot mọi người chat trước khi buổi workshop này

ALEX
- 2

App hoạt động thế nào

Một app gồm những gì, API call là gì, vibe coding là gì

TRIẾT
- 3

Chọn lựa topic + plan 2 tuần tới

Chủ đề, nhóm, cách làm, và output cuối cùng

ALEX

MỤC TIÊU SAU 2 TUẦN



1

Tự build được app/chatbot/landing page và ngay cả mini-game

Mình nói bằng tiếng Việt, AI viết code. Prompt nhỏ từng phần, hỏi trade-off, bắt giải thích lại tới khi hiểu.



2

Nối app với model AI

Call API là gì. Chọn model phù hợp cho việc của mình, đôi khi không phù hợp. Xử lý khi model lỗi.



3

Làm việc trên code chung

Đưa code lên GitHub. Lấy code sẵn có của người khác làm nền cho app của mình.



4

Thiết kế trước khi làm

Vẽ được sơ đồ app của mình. Chia một ý tưởng lớn thành từng phần làm được.



5

Tự kiểm tra và kiểm soát đầu ra của AI

Test bằng ví dụ thật của mình. Nhận ra khi có thứ báo "thành công" mà đang hỏng.

OUTPUT

Một app nhóm mình tự làm, người khác dùng được — và nhật ký ghi lại cách mình làm ra nó.

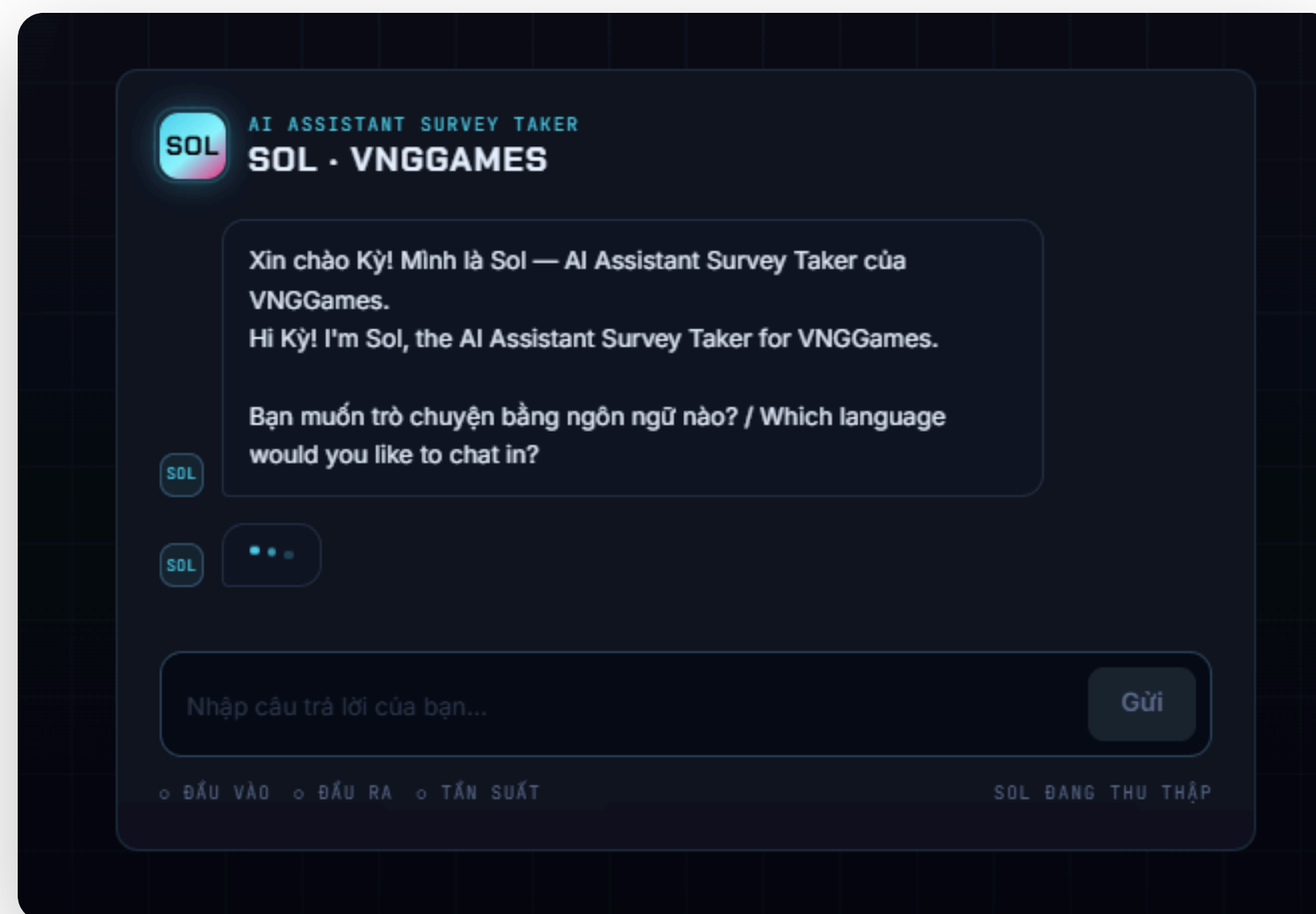
PHẦN 1

VÌ SAO AI CŨNG HỌC ĐƯỢC

Không phải lý thuyết. Đây là chuyện mình vừa làm tuần trước.

MỌI NGƯỜI ĐÃ DÙNG THỬ RỒI

Con bot survey tuần trước là một mini-app. Không phải Google Form.



🔗 [Bấm để xem Sol làm được gì](#)

Mỗi người trả lời theo một kiểu khác nhau — form không xử lý được chuyện đó.

TRẢ LỜI CHO VUI	<i>"dạ hong á" · "abc" · "mất khoảng 10p =))))" · "ai mà bíc má, tùy á"</i>	Sol vẫn chốt được: upload file ghi âm cuộc họp → bản tóm tắt + việc cần làm.
MUỐN QUÁ NHIỀU	<i>"a và b luôn nha" — rồi thêm: phân loại theo dự án, sắp ưu tiên, alert khi gần hạn</i>	Sol nói thẳng cái nào không làm được trong 2 tuần , rồi đưa 2 phương án hẹp hơn để chọn.
KỂ RẤT CHI TIẾT	<i>Cả quy trình từ nhận job requisition tới close job — một đoạn dài 10 dòng</i>	Sol chỉ ra 2 bước có thể làm thành mini-app , và bắt chọn đúng một bước.
CHƯA BIẾT MUỐN GÌ	<i>"bạn có thể gợi ý kg" · "chưa"</i>	Sol đổi hướng gợi ý hai lần , tới khi chốt được (a) — dán ghi chú họp → tóm tắt + đầu việc.
SỬA LẠI BOT	<i>"không phải tóm tắt, một bài đăng social"</i>	Sol nhận sai và viết lại spec đúng ý : brief Word → bài post hoàn chỉnh, dịch Anh + Trung.

Một cái form chỉ nhận được đúng những gì nó hỏi. Năm kiểu người ở trên đều **ra được một thông tin cụ thể vào vấn đề và dùng được** — đó là lý do phải làm chatbot, không phải Google Form.



✂ Bấm để xem thực tế mất bao lâu

THỨ SẼ LÀM MỌI NGƯỜI MẤT NHIỀU THỜI GIAN NHẤT

Đừng tin 100% những gì nó báo "thành công"

- ⚠ Báo chạy thành công — app chết ngay khi vừa mở lên
- ⚠ Báo cập nhật thành công — nhưng vẫn đang chạy bản cũ, code mới chưa lên
- ⚠ App gặp lỗi nhưng im lặng, không báo gì — chỉ thấy kết quả trống
- ⚠ Trang admin báo "sai mật khẩu" — trong khi mật khẩu hoàn toàn đúng

QUY TẮC

Đừng tin dòng chữ "thành công". Hãy tự kiểm tra xem thứ mình cần đã thay đổi thật chưa.

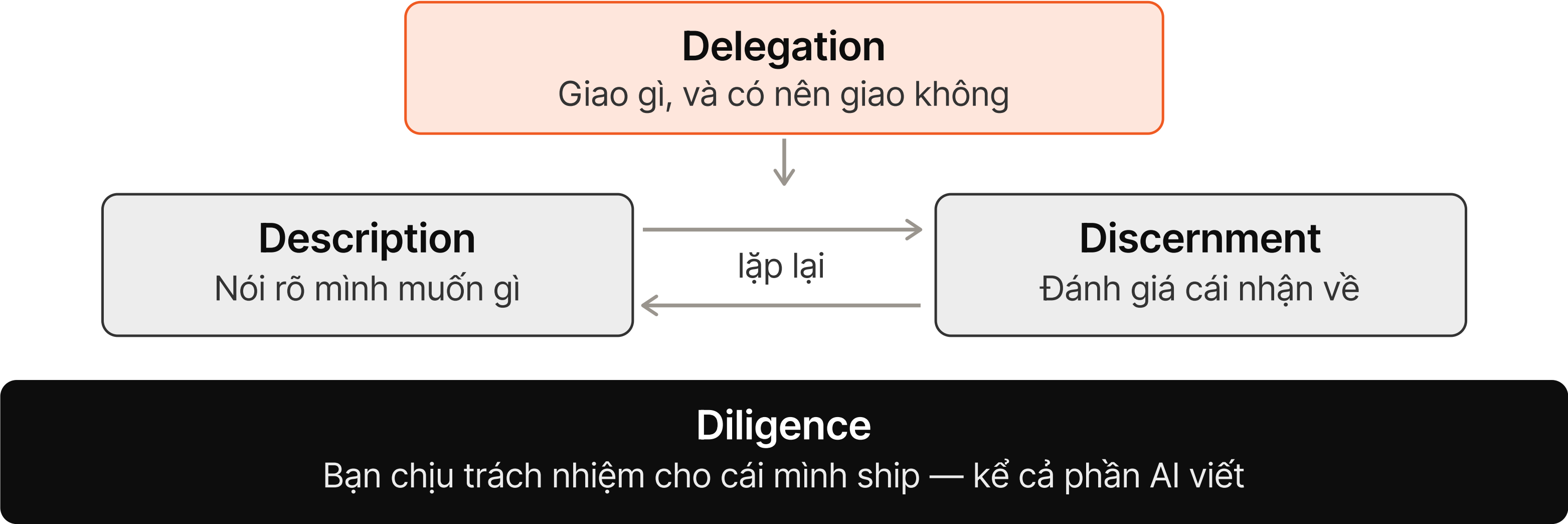
Việc này không cần biết code. Chỉ cần tự mở app ra dùng, và chỉ ra đúng chỗ mình thấy sai.

PHẦN 2

APP HOẠT ĐỘNG THẾ NÀO

Khung 4D, vai trò, Git, và thói quen.

Bốn việc bạn vẫn phải tự làm



Description và **Discernment** là vòng lặp mình làm mỗi ngày. **Diligence** là cái nền — không giao cho ai được.

Một SaaS có khách trả phí, đóng vĩnh viễn trong một tuần

APP GÌ, PHỤC VỤ MỤC ĐÍCH GÌ

Một SaaS tìm khách hàng tiềm năng. Khách gắn một đoạn mã lên website của mình; app nhận diện người vào xem và trả về thông tin liên hệ cho bên bán hàng gọi. App có khách trả phí thật.

BỐI CẢNH

- Non-Tech Builder
- Làm một mình
- Không có reviewer
- Ship thẳng cho end-user

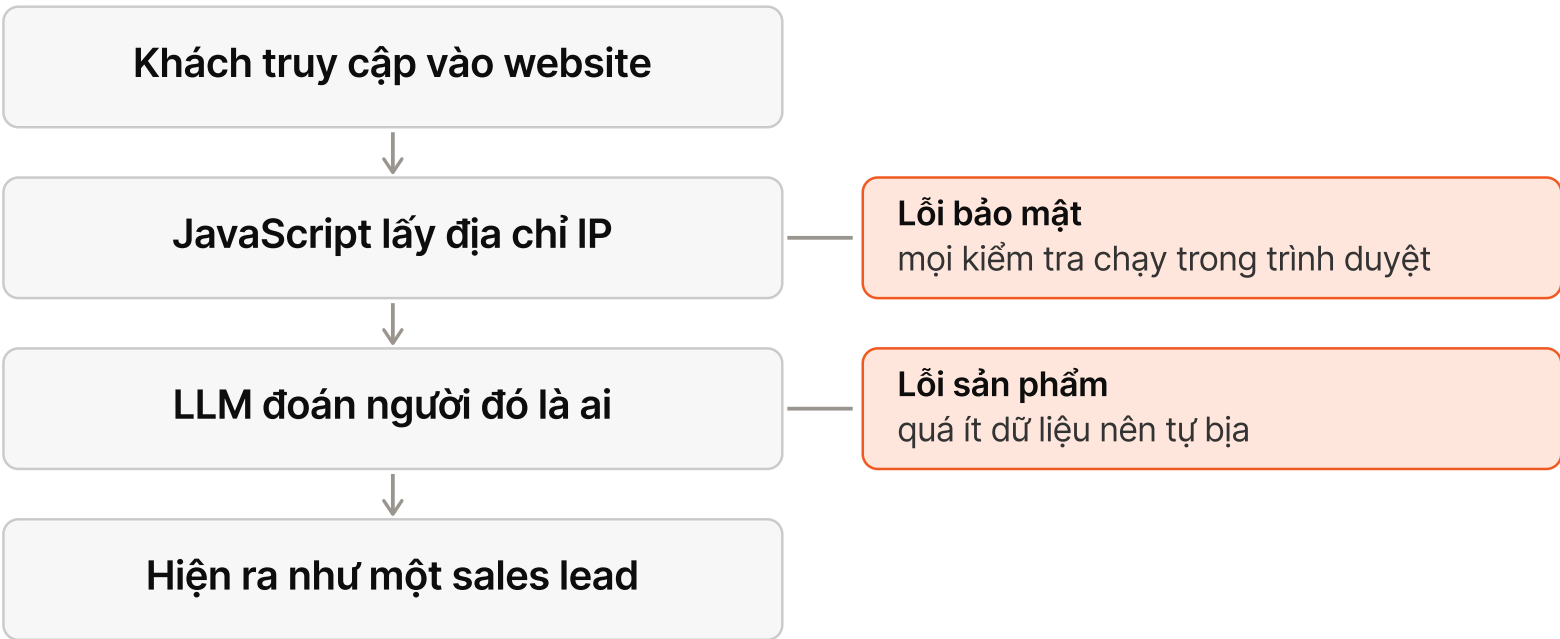
SAI LẦM & ROOT CAUSE

Mọi lớp kiểm soát nằm ở phía trình duyệt người dùng, không ở phía server. Khoá dịch vụ nằm ngay giao diện người dùng.

- **Discernment** — chỉ nghiệm thu bằng câu hỏi "app chạy được chưa", không kiểm thử ở vai trò người dùng
- **Diligence** — ship cho khách trả phí mà không có bất kỳ giải pháp bảo mật nào

HẬU QUẢ

Hai ngày sau bài đăng: bản trả phí bị dùng miễn phí, dữ liệu rác tràn vào database. Nhờ chính AI sửa thì mỗi lần sửa lại hỏng thêm chỗ khác.



Lesson learned — Delegation: anh đã giao phần bảo mật cho AI mà không biết là mình đã giao. AI xây cái bạn mô tả, nó không cảnh báo cái bạn quên hỏi.

Thành phần của app không đổi. Chỉ người phụ trách là đổi.

Thành phần	Mô hình truyền thống	Mô hình App owner
Màn hình	Frontend Developer	App owner AI
Logic xử lý	Backend Developer	App owner AI
Database	Data Engineer, DBA	App owner IT
Đăng nhập	Platform Engineer	Owner IT
Hosting	DevOps, SRE	IT
Kiểm thử	QA Engineer	App owner
Review bảo mật	Security Engineer	Không ai nhận

App owner AI IT Không ai

Trong SDLC truyền thống, việc tách vai không chỉ để chia việc — chính nó là cơ chế kiểm soát chất lượng.

MINI-APP

Bạn phù hợp môn thể thao nào

Một mini-app được vibe-code trong vài giờ — trả lời vài câu, nó gợi ý môn thể thao phù hợp với bạn.

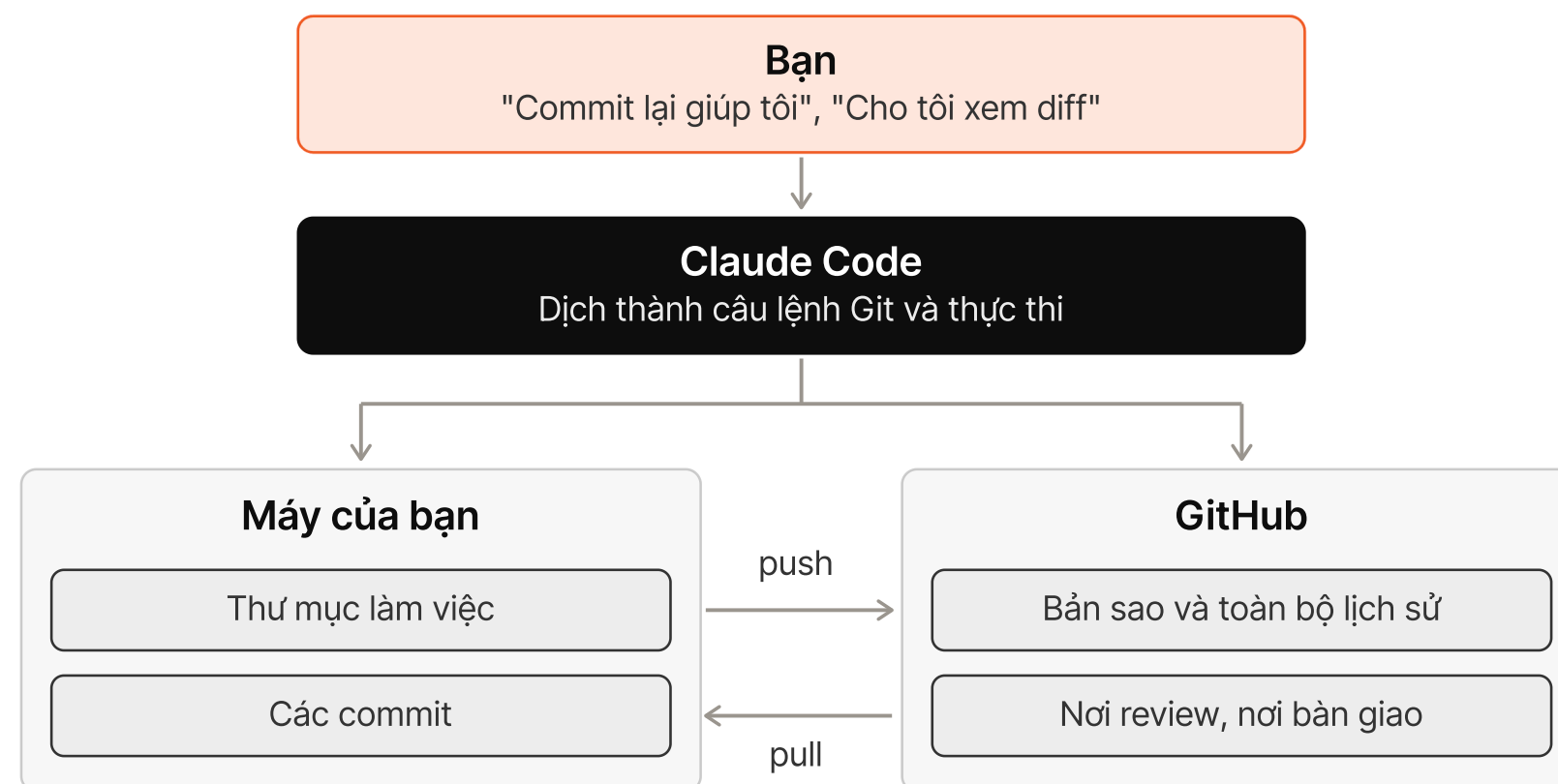
CÁCH 1 Quét mã QR bên cạnh

CÁCH 2 sport-match.trietchm.workers.dev



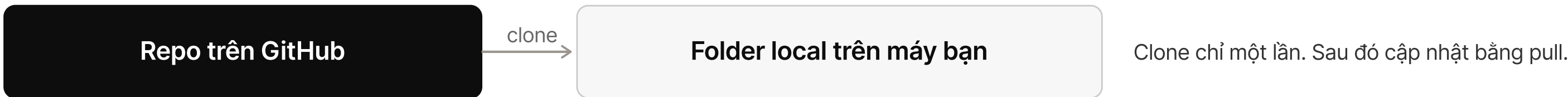
GIT VÀ GITHUB LÀ HAI THỨ KHÁC NHAU

- 🕒 **Git** là lịch sử phiên bản của một thư mục
- ☁️ **GitHub** là bản sao thư mục đó nằm trên internet
- >_ Bạn không gõ câu lệnh Git — Claude Code gõ.
Nhưng bạn cần biết hai vùng này khác nhau



Vòng làm việc — spec, build, rồi quay lại

Chuẩn bị một lần



Vòng lặp mỗi tính năng



BẢY KHÁI NIỆM CẦN BIẾT

KHÁI NIỆM	NGHĨA	VÌ SAO CẦN BIẾT	NÓI VỚI CLAUDE CODE
Repository (repo)	Thư mục dự án, kèm toàn bộ lịch sử của nó	Đơn vị bạn sở hữu và bàn giao	"Tạo repo mới cho dự án này"
Clone	Tải một repo về máy, kèm toàn bộ lịch sử	Cách lấy app có sẵn về để chạy, học, hoặc tiếp tục việc người khác	"Clone repo này về và giúp tôi chạy nó"
Commit	Một điểm lưu, kèm ghi chú tại sao	Nút hoàn tác của bạn — không có commit thì không có gì để quay về	"Commit lại trước khi bắt đầu"
Push / Pull	Đẩy các điểm lưu lên GitHub, hoặc lấy thay đổi mới về	Chưa push thì việc chỉ tồn tại trên máy bạn	"Push lên GitHub giúp tôi"
Branch	Bản song song để thử; xong thì nhập lại vào bản chính (main)	Chỗ để Claude làm mạnh tay mà không đụng bản đang chạy	"Tạo branch mới để thử tính năng này"
Diff	Danh sách những gì đã thay đổi	Thứ duy nhất bạn review được mà không cần đọc code	"Cho tôi xem diff, tóm tắt đã đổi những gì"
Revert	Quay về một điểm lưu trước đó	Cái làm việc thử nghiệm với AI trở nên an toàn	"Quay lại commit trước"

3 THÓI QUEN NÊN LÀM QUEN VỚI VIBE-CODING



Spec là source of truth

Mỗi thay đổi bắt đầu bằng sửa spec, rồi mới nhờ AI làm.



Yêu cầu AI giải thích lại cái nó vừa xây

Sau mỗi tính năng lớn: *"Giải thích bằng ngôn ngữ mình hay nói chuyện với nhau: gồm những phần nào, mỗi phần làm gì."*



Kiểm chứng bằng tay theo tiêu chí viết trước

Tiêu chí nghiệm thu phải là câu bạn tự bấm thử được.

BA THÓI QUEN CẦN BỎ



Mặc định chọn phương án "recommended"

Ba câu trước khi bấm đồng ý: *"Vì sao chọn phương án này?" · "Hai phương án kia đánh đổi cái gì?" · "Bạn đang giả định điều gì mà tôi chưa nói?"*



Hỏng rồi vẫn sửa tiếp, thay vì quay lại

Hai lần sửa không xong thì quay về commit trước và mô tả lại theo cách khác.



Dán khoá và mật khẩu vào cửa sổ chat

Khoá nằm ở secret store do bên IT cấp, không đi qua chat.

Không phải "đừng tin AI". Phương án recommended thường hợp lý. Vấn đề là chọn mà không biết mình vừa đánh đổi cái gì.

POP QUIZ — 10 CÂU

Trước khi qua Phần 3, kiểm tra lại các thuật ngữ vừa học

Quét QR, làm 10 câu về những thuật ngữ cần nhớ để vibe-code: **spec**, **branch**, **commit**, **push**, **pull**, **merge**, và các khái niệm Git khác.



PHẦN 3

CHỌN LỰA TOPIC & PLAN 2 TUẦN TỚI

Chủ đề, nhóm, cách làm, và thứ phải nộp cuối cùng.

Ý tưởng mọi người gần như giống nhau

8 người

hoàn thành survey

6 ý tưởng

xử lý text dán vào

2 ý tưởng

đọc file doc / PDF

CẢ 8 Ý TƯỞNG ĐỀU CHUNG 1 XU HƯỚNG

Thứ lộn xộn mình đang có → thứ gọn gàng mình cần

Tám người, tự trả lời riêng, không ai bàn với ai — và cùng mô tả đúng một kiểu app.

4 NHÓM — CHỌN 1 TRONG 6 CHỦ ĐỀ DƯỚI, ĐƯỢC QUYỀN TINH CHỈNH

NHÓM 1Kiểu + Tiến

BASELINE

JD/CV → câu hỏi phỏng vấn

Dán JD vào, ra bộ câu hỏi phỏng vấn phù hợp. Dán thêm CV thì ra bản đánh giá độ phù hợp kèm câu hỏi cần đào sâu.

Mechanic: một input, một lần call API, output là danh sách có cấu trúc. Không có hội thoại, không có nhiều lượt — đơn giản nhất trong các đề tài.

Khó ở đâu: câu hỏi ra sẽ chung chung, kiểu nào cũng dùng được cho mọi vị trí. Muốn nó ra câu hỏi riêng cho đúng JD đó thì phải nói cho model biết thế nào là một câu hỏi tốt ở VNGG — đó là phần prompt, không phải phần code.

NHÓM 2Dung + Thương Thương

BASELINE

Ghi chú họp → việc cần làm

Dán ghi chú họp hoặc ghi chú cuộc gọi vào, ra bản tóm tắt kèm danh sách việc cần làm, ai làm, hạn nào.

Mechanic: một input, một lần call API, output là bảng có cột. Cũng là đề tài dễ nhất về mặt kỹ thuật.

Khó ở đâu: model không phân biệt được "cái này team quyết rồi" với "cái này mới chỉ bàn tới". Nó sẽ biến mọi câu nói thành một việc cần làm. Dạy nó phân biệt hai thứ đó là toàn bộ công việc của 2 tuần.

NHÓM 3Thảo + Vy

ADVANCE

Phỏng vấn với AI

Dán JD; app trở thành ứng viên cho vai trò đó và bạn phỏng vấn nó, hoặc nó phỏng vấn bạn.

Mechanic: roleplay nhiều lượt với một persona.

Khó ở đâu: persona trôi khỏi vai, và không biết khi nào nên dừng.

ADVANCE

Máy đổi giọng văn

Nhập một bài viết social draft; nhận cùng lúc ba phiên bản: chuyên nghiệp / trẻ trung / hài, cộng một cặp Việt ↔ Anh.

Mechanic: một input, nhiều output đặt song song.

Khó ở đâu: làm ba bản thực sự khác nhau — buộc phải đưa một bảng thuật ngữ vào prompt.

ADVANCE

Have A Sip với AI

Bạn là khách mời. Hai AI cùng nói chuyện với bạn — lúc đồng ý với nhau, lúc phản bác nhau, lúc phản bác bạn.

Mechanic: hai persona trong một cuộc hội thoại.

Khó ở đâu: mặc định cả hai sẽ đồng ý với nhau vì lịch sự là bản năng được train. Mỗi bên cần một lập trường đối lập rõ ràng trong prompt — một tối ưu cho sự thoải mái của bạn, một tối ưu cho sự phát triển — hoặc lượt gọi thứ hai phải được yêu cầu tìm lỗi của lượt đầu. Thiếu vậy sẽ ra hai giọng nói cùng một điều, nghe ầm ập nhưng tệ hơn một giọng.

ADVANCE

Nổi tử với AI

Game nổi tử tiếng Việt nhiều người chơi, AI là một trong những người chơi.

Mechanic: chơi theo lượt, AI tới lượt thì cũng phải nổi như mọi người. Luật do nhóm tự set — nổi theo âm cuối, hoặc biến thể khác. Ai thấy tử nào lạ thì challenge; challenge sai thì mất điểm.

Khó ở đâu: model rất hay trả về tử không bắt đầu bằng đúng âm cần nổi, hoặc bịa ra tử không có thật. Nên phải để code kiểm tra từng tử AI đưa ra, sai thì bắt nó trả lời lại.

CÁCH LÀM — 5 BƯỚC, LẶP LẠI

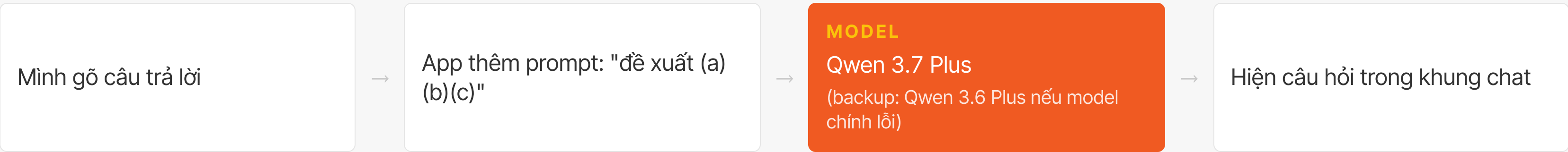
0	Break down 1 function, 1 mục tiêu, nói được trong 1 câu	Survey Chatbot Một bộ câu hỏi, mỗi câu có mục tiêu riêng, và thông tin cần bóc ra từ mỗi câu.
1	Reality check Cái này có nên làm không? Có cách nào tốt hơn?	Survey Chatbot Đầu tiên nghĩ làm survey kiểu Google Form. Nhưng nhạt — muốn nó tương tác được với mọi người, nên chuyển sang chatbot.
2	Clarify goal Input gì, output gì, AI cần biết thêm gì	Survey Chatbot Input: người ta gõ gì. Output: ý tưởng gồm input – output – tần suất.
3	Spec → Build Prompt nhỏ. Hỏi trade-off. Không hiểu thì bắt giải thích một cách siêu đơn giản cho học sinh tiểu học	Survey Chatbot Chia thành 9 task. Task 3 mới là task đầu tiên có call API — 2 task trước chỉ là màn hình và chỗ lưu data.
4	Test Tự chạy full flow. 1 ví dụ ok — còn 5, 10, 20 ví dụ khác nhau thì sao?	Survey Chatbot Tự chat thật bằng tiếng Việt. Phát hiện: bot cắt lời ở câu 4, và câu hỏi tối nghĩa không ai trả lời nổi.

SƠ ĐỒ MẪU CỦA SURVEY CHATBOT

MỤC TIÊU — NÓI ĐƯỢC TRONG 1 CÂU

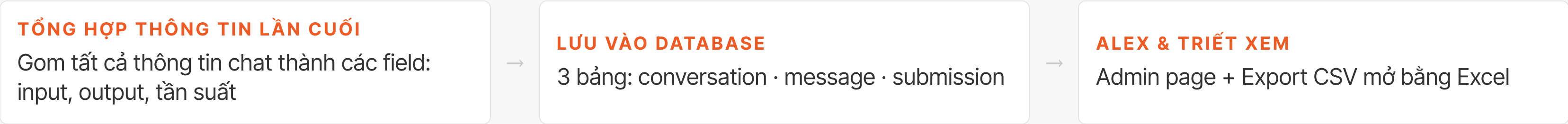
Lấy đủ thông tin từ mỗi người để chốt ra được các project cho 2 tuần tới.

VÒNG HỘI THOẠI



↺ lặp lại tới khi Sol thấy đủ thông tin

KHI CHAT XONG



4 QUY TẮC KHÔNG THƯƠNG LƯỢNG

- 

1

Chưa vẽ sơ đồ thì chưa gõ prompt

Vẽ ngay hôm nay, trước khi mở Claude Code. Draft cũng được.
- 

2

Prompt nhỏ ngay từ đầu

Đừng lặp lại lỗi cơ bản: "build cho tôi cả cái app".
- 

3

Tự test bằng ví dụ thật của mình

Không phải ví dụ đẹp mình dùng lúc build.
- 

4

Ghi lại những gì mình chưa hiểu

Claude tự ghi diary hộ — nhưng dòng "chưa hiểu" phải tự tay viết.

Quy tắc số 4 nghe đơn giản nhất, nhưng đó lại là **thứ mình quan tâm nhất** khi mọi người trình bày ở buổi 2.

NGAY SAU ĐÂY — MỖI NHÓM BẮT TAY VÀO VIỆC

Không về nhà mới làm. Bắt đầu tại chỗ, còn Alex & Triết ngồi cùng.


1
Ngồi lại với nhau
Hai người trong nhóm chốt một ý tưởng chung.


2
Vẽ sơ đồ
Draft cũng được, vẽ tay cũng được. Miễn là chỉ được vào từng hộp và nói nó làm gì.


3
Chia task
Ai làm phần nào. Ghi ra giấy luôn, đừng để "rồi tính sau".


4
Mở Claude Code
Bắt đầu ngay tại chỗ. Vướng gì thì gọi Alex hoặc Triết.

Alex & Triết sẽ **chụp lại sơ đồ của từng nhóm hôm nay**. Đó là bản gốc để đối chiếu ở mỗi checkpoint.

☰ **Bấm để xem lại 6 chủ đề**

2 TUẦN TỚI — 2 CHECKPOINT MỖI TUẦN

Tuần 1

Build theo task đã chia. Cuối tuần: dán input vào, có output hiện ra. Xấu cũng được.



Tuần 2

Đưa người nhóm khác dùng thử — không giải thích, chỉ ngồi xem. Rồi sửa đúng chỗ họ vướng.



Buổi 2

Trình bày trước cả team.

CHECKPOINT — 2 LẦN MỖI TUẦN, 10 PHÚT

Alex hoặc Triết đọc diary của nhóm trước, rồi ngồi xuống hỏi đúng 4 câu:

1

Làm được tới đâu rồi?

Cho xem app chạy, đừng kể

2

Đang ở bước nào trong 5 bước?

Spec, build, hay đang test

3

Đang vướng gì?

Mang bug tới, mình gỡ cùng

4

Kế hoạch tiếp theo?

Câu quan trọng nhất

Mỗi checkpoint, mình mở lại sơ đồ gốc của nhóm — có gì đổi không? **Chính cái thay đổi đó là tiến độ.** Từ sơ đồ rough hôm nay tới bản cuối cùng, đó là thứ nhóm sẽ kể ở buổi 2.

Không phải chép tay. Làm xong, gõ `/recap` là xong. Khoảng 20 giây.

DIARY.MD

09/09 – Kỳ

Nhờ làm: màn hình dán file vào + nút bấm
Claude làm: xong, bấm nút thì hiện tên file ra
Bước: 3 – Spec → Build
Chưa hiểu: sao phải tách 2 phần, một phần
chạy trên máy mình một phần không



Claude tự ghi

Mình nhờ gì, nó làm ra cái gì, đang ở bước nào trong 5 bước.



Mình viết 1 dòng

Chỗ chưa hiểu hôm nay. Claude sẽ hỏi lại — chỉ cần trả lời.



Claude tự nhắc

Đầu mỗi session, nếu hôm trước chưa ghi thì nó nhắc một câu.

Viết 20 giây mỗi ngày, để checkpoint 10 phút dành cho **việc gỡ vướng** — không mất vào kể lể. Và tới buổi 2, phần "cái gì mình chưa hiểu" đã có sẵn — không phải ngồi nhớ lại.

Đây là thứ phải nộp. Không phải bản báo cáo, không phải slide đẹp.

2 phút **Người nhóm khác dùng app của bạn, ngay tại chỗ**
Không phải bạn bấm. Người chưa từng dùng bấm.

1 phút **Sơ đồ lúc đầu, và sơ đồ bây giờ**
Cái gì đã đổi so với bản vẽ hôm nay, và tại sao lại đổi.

2 phút **Cái gì hỏng, và cái gì bạn vẫn chưa hiểu**
Lấy thẳng từ dòng "chưa hiểu" trong diary. Không ai phải che chuyện thất bại.

Definition of done không phải là "nó chạy được trên máy mình" — mà là **người khác dùng được mà không cần mình chỉ.**

VIỆC CỦA MÌNH

**Không phải viết code. Là nhận ra khi nào có
gì đó chưa đúng.**

Và cái đó thì không cần học code.

Hẹn gặp mọi người ở buổi 2. · Alex & Triết